

Stylus Mini-Apps on Arbitrum

Final report: BananitaSwap, CoinFlip and Vault Aggregator. Insights, qualitative findings on Stylus performance, integration patterns and mainnet-readiness.

WakeUp Labs · three Stylus mini-apps on Arbitrum: **BananitaSwap** (USDC↔token swap through Camelot and Uniswap V3, Stylus router, hosted in Lemon), **CoinFlip** (Stylus, Chainlink VRF) and **Vault Aggregator / Vaulty** (Stylus, four-protocol USDC yield router), plus the cross-cutting Stylus findings the last two produced. Every figure traces to a source document; where a number was never recorded, this report says so. No figure here is estimated.

Contents

1 Executive summary

2 BananitaSwap (token swap on Arbitrum, hosted in Lemon)

2.1 Architecture

2.2 Buy / sell, and the single-signature batch

2.3 Status

3 CoinFlip (Stylus, Arbitrum)

3.1 Architecture

3.2 Bet → VRF → payout, and the refund fence

3.3 Permit2 and ERC-1271

3.4 Status

4 Vault Aggregator (Vaulty)

4.1 Architecture

4.2 Deposit → yield → two-step withdraw

4.3 Status: production on Arbitrum One

5 Stylus performance findings

5.1 The real mainnet gate: one fragment

5.2 Size levers, measured

5.3 Activation cost

5.4 Gas per operation

5.5 eth_estimateGas under-reports Stylus→EVM calls

5.6 Randomness funding: native ETH over LINK

6 Integration patterns

6.1 Chainlink VRF v2.5 direct funding from Stylus

6.2 sol_interface! computes the wrong selector for struct-shaped calldata

6.3 TestVM's single return-data buffer

6.4 Permit2 from Stylus, and ERC-1271 signatures

7 Mainnet-readiness

- 7.1 The pipeline that worked
- 7.2 The pipeline's open problem: verification of -Zbuild-std builds
- 7.3 Fork testing: the state of the tooling
- 7.4 Immutable owner, by design
- 7.5 Recommendations to the Stylus / Arbitrum team

1 Executive summary

Ten Stylus (Rust) contract binaries ship across three systems (thirteen deployed instances, since Vaulty's adapter is deployed once per protocol): a swap router with two DEX adapters, a VRF-backed coin-flip game and a USDC yield aggregator that splits one deposit across four real Arbitrum lending protocols.

BananitaSwap's Stylus router and its Camelot and Uniswap V3 adapters are live on Arbitrum One at `0x0e2873AC929e5C6fFD15EA3D9c8CD2ec6cec9Ad0`, fee set to 0. **Vaulty is LIVE on Arbitrum One since 2026-08-10**: `vault-core 0x8a1758d3dd3d1049c43bfb1d1fec11fd403d3553` plus a periphery and four adapters on real Aave, Morpho, Fluid and Euler vaults, smoke-tested the same day with real USDC at a total cost of 7 units of ERC-4626 round-down (7 millionths of a USDC). **CoinFlip is live on Arbitrum Sepolia**, core, token and faucet Arbiscan-verified, the ERC-1271 periphery live and QA'd end to end; its Arbitrum One deploy was deferred by client decision.

Five findings generalize beyond these apps (measured on CoinFlip and Vaulty):

1. **The operative mainnet WASM gate is one fragment, not just 24 KB.** Arbitrum One's ArbOS has no multi-fragment support while Sepolia's does: the same 2-fragment core passed `cargo stylus` check against Sepolia and failed against Arbitrum One. Checking against Sepolia alone silently passes binaries that cannot activate on mainnet.
2. **The panic strategy is the largest size lever by an order of magnitude** — `-Cpanic=immediate-abort + -Zbuild-std` reclaimed ~9.9 KB, against ~2.1 KB for every feature relocation combined. Pull it first; it took CoinFlip's core from 32,314 bytes / 2 fragments to 23,761 bytes / 1 fragment.
3. **`eth_estimateGas` under-reports Stylus→EVM calls.** Two live transactions sent at the estimator's limit mined out-of-gas, and the same calls retried with headroom succeeded using less gas than that limit; production clients pin 6,000,000.
4. **Solidity SDKs hide external plumbing behind the hook you implement, and Stylus has no inheritance to hide it for you.** Chainlink direct funding calls back from the *wrapper*, so validating against the coordinator rejects every fulfillment. Verify every callback selector with `cast sig` before deploying.
5. **There is no fork-testing story for Stylus against live L2 state.** The substitute was small real-USDC transactions on Arbitrum One.

2 BananitaSwap (token swap on Arbitrum, hosted in Lemon)

2.1 Architecture

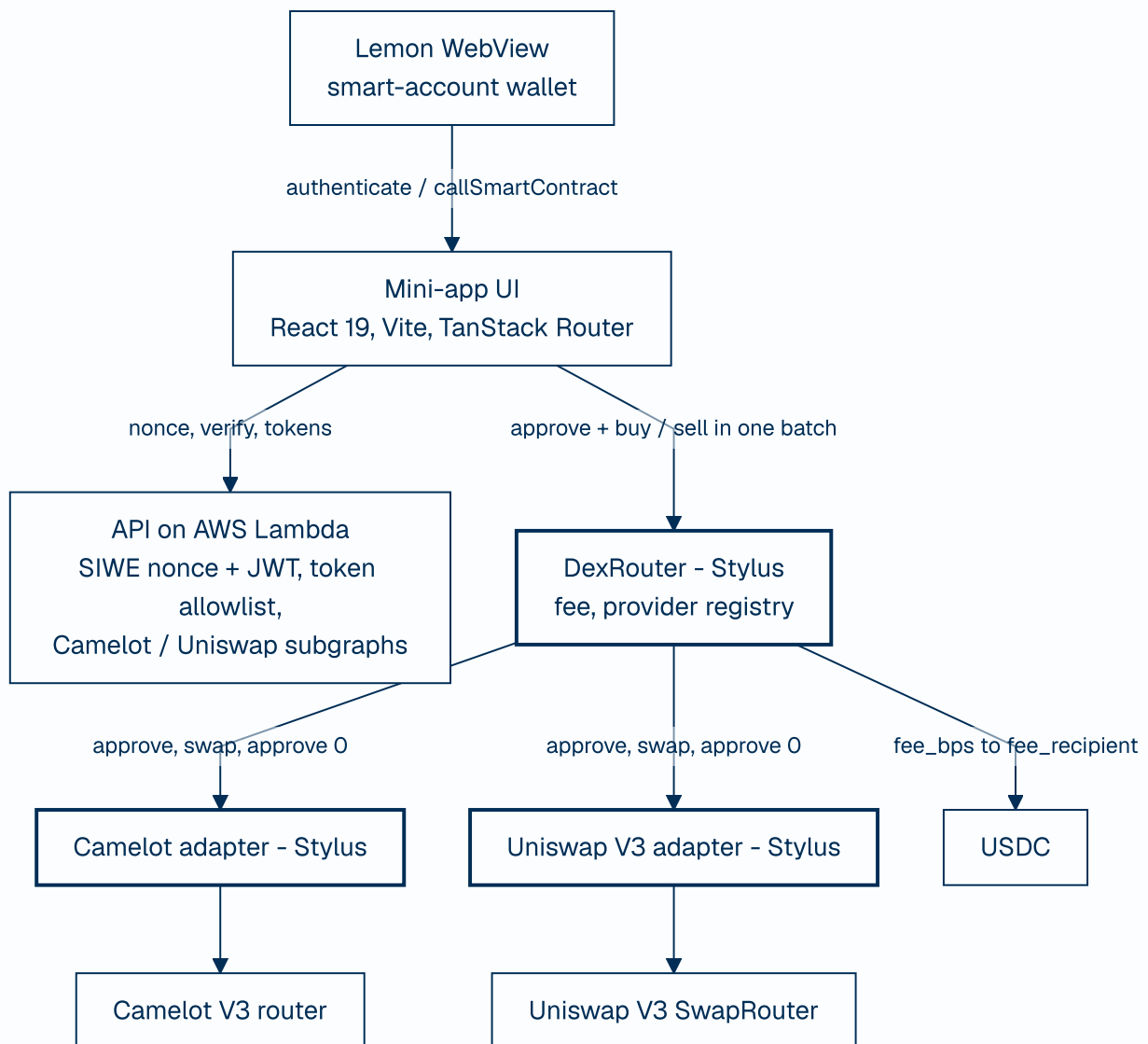


FIGURE 1 · ARCHITECTURE

A USDC-in, token-out swap mini-app: the user picks one of the allowlisted Arbitrum tokens, buys it with USDC or sells it back, and the route goes through Camelot V3 or Uniswap V3. The repository holds four packages with no npm workspaces: `ui` (React 19.2, Vite 6, TanStack Router, viem 2.46, @lemoncash/mini-app-sdk 0.1.15), `api` (Hono 4.12 + Drizzle on Neon Postgres, deployed as an AWS Lambda behind API Gateway by SST v3), `sol-contracts` (an EIP-2535 Diamond router

in Solidity ^0.8.24, Hardhat) and contracts, a **Stylus router in Rust** on stylus-sdk 0.10.2 with the nightly-2025-07-01 toolchain and `opt-level = "s"` (CoinFlip and Vaulty, pinned months later, are on stylus-sdk 0.10.7 and stylus-test 0.10.8).

The Stylus side is three small contracts selected by Cargo feature (`contract-router`, `contract-camelot`, `contract-uniswap-v3`), 494 lines of Rust in total. `DexRouter` stores `owner`, `usdc`, `fee_bps`, `fee_recipient` and a `provider_id → adapter` map; `initialize(usdc, fee_bps, fee_recipient)` runs once and makes the caller owner, `buy/sell` take a u8 provider id, and the owner can `setAdapter/removeAdapter`, change the fee (capped at 10,000 bps) and `transferOwnership`. Adapters are **separate contracts called externally**, never `delegatecall`: the router approves the adapter, calls `IDexAdapter.swap(tokenIn, tokenOut, amountIn, minOut, deadline)`, and resets the approval to zero. Adding a DEX therefore means deploying an adapter and one `setAdapter` call, with no upgrade path for the router itself.

The Solidity Diamond is the parallel implementation: `RouterFacet.buy/sell` measure the pulled amount by balance delta, deduct the fee and `delegatecall` a `CamelotFacet` or `UniswapV3Facet`, both single-hop `exactInputSingle` with no on-chain quoter, behind bytes32 provider ids and full `diamondCut` upgradeability under a single owner. The frontend's router ABI is the Stylus one (`buy(address,uint256,uint256,uint8,uint256)`, `getUsdc()`, `getAdapter(uint8)`), and the UI env template names the variable "Rust/Stylus Diamond Router address per network".

2.2 Buy / sell, and the single-signature batch

Authentication follows the Lemon SDK: the app fetches a nonce from the API, calls `authenticate`, and posts the returned SIWE message and signature to `POST /auth/verify`, which validates it with `viem/siwe` (ERC-1271 capable) and issues a JWT. If backend verification fails the wallet stays in state without a JWT, so on-chain reads and deposit/withdraw keep working.

A swap reads `getUsdc()` off the router, resolves the provider id (Camelot = 1, Uniswap = 2), takes an on-chain quote, then simulates with an escalating slippage ladder from 50 to 1,000 bps, retrying only on "too little received". When the allowance is short, the ERC-20 approve and the router `buy/sell` go into a single `contracts[]` array for one `callSmartContract`, so Lemon signs one atomic batch instead of two prompts (PR #31, 2026-05-27). The same hook detects Lemon whitelist rejections and tells the user which contract needs allowlisting. The API serves an allowlist of **116 curated Arbitrum One tokens** built from The Graph subgraphs of Camelot V3 and Uniswap V3.

2.3 Status

Arbitrum One — Stylus router live, read on-chain on 2026-08-17:

CONTRACT	ADDRESS	OBSERVATION
DexRouter (Stylus)	0x0e2873AC929e5C6fFD15EA3D9c8CD2ec6cec9Ad0	code present; <code>getFeeBps()</code> = 0; owner 0x9217bB4Af4429EB4D61a78b65B99B3688723Ab86

CONTRACT	ADDRESS	OBSERVATION
Camelot adapter (Stylus), id 1	0x11A92F52db0758789b3EF426CBF a52286736a9d7	registered via getAdapter(1)
Uniswap V3 adapter (Stylus), id 2	0xd7E120DDD3a96a46621BC77e982 bc3Cd6EAdFe4F	registered via getAdapter(2)
Diamond router (Solidity)	0x1eef9f0918c61fe2808c65a7ca5 4998000f480da	code present; earlier/parallel implementation

The table reports on-chain state. On Arbitrum Sepolia the committed Diamond constant points at a mock deployment with mock WETH/USDC; the mainnet Diamond, its facets and the real Camelot V3 router and USDC addresses are documented in the contracts README.

The app was built between 2026-02-06 and 2026-06-09: 128 commits and pull requests #1 through #37, with the Solidity and Rust contracts merged on 2026-04-01 (PRs #1 and #2) and the last cycle (on-chain quotes, the top-token allowlist) in early June. CI builds the API and UI on pull requests and deploys the staging stage on push to main through SST into AWS; a demo video is linked from the README.

The Stylus size, activation and gas findings in chapters 5 and 6 come from CoinFlip and Vaulty, which kept per-transaction records throughout.

3 CoinFlip (Stylus, Arbitrum)

3.1 Architecture

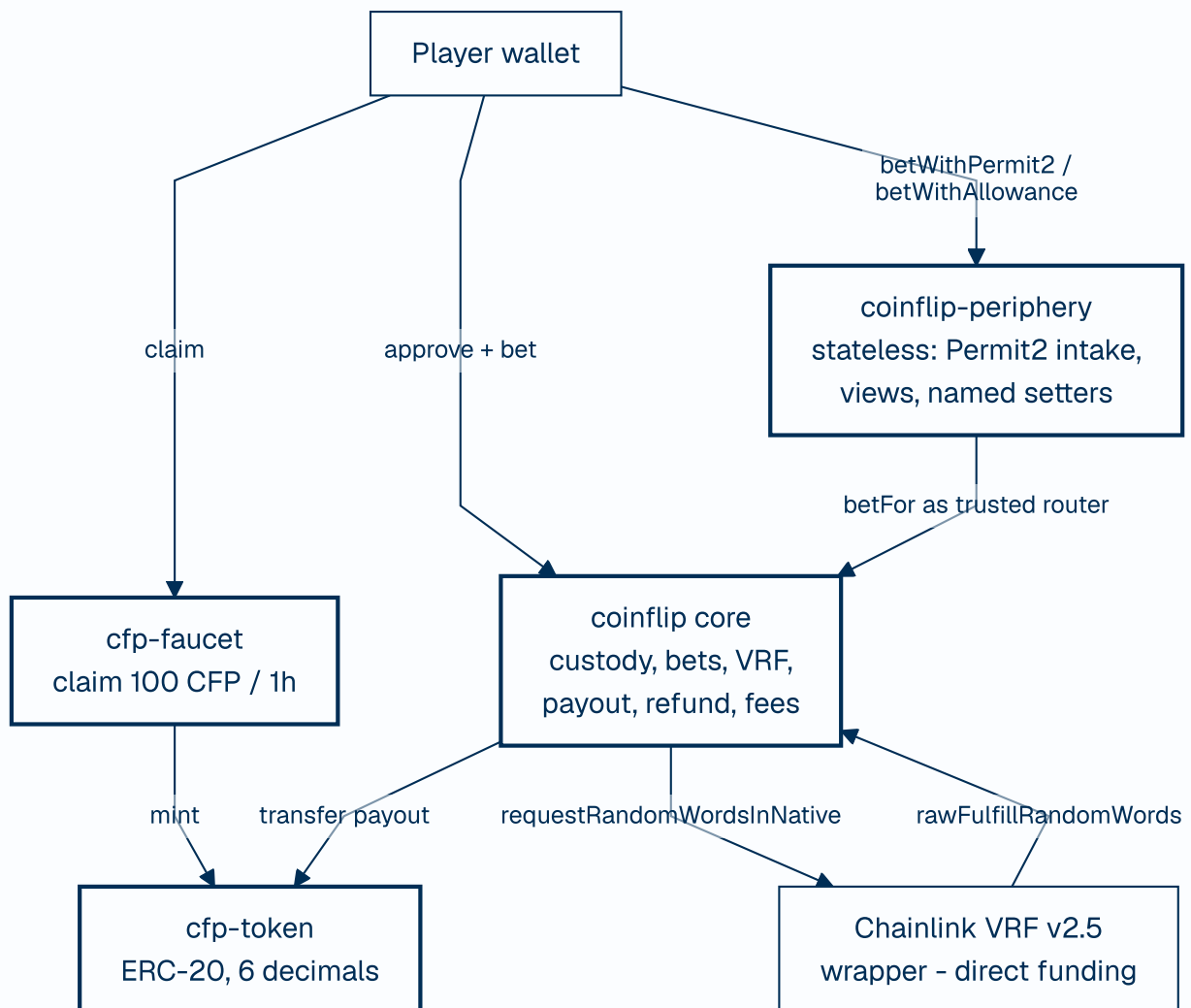


FIGURE 2 · ARCHITECTURE

Four contracts, one custody point. The **core** owns every game-state transition and holds the Chainlink VRF v2.5 direct-funding integration inside it, behind a `RandomnessSource` trait; the **periphery** is stateless and ownerless, pulling tokens through Permit2 with the destination set to the core, and the core never trusts it for money-moving invariants. `cfp-token` is 6-decimal play money, minted by `cfp-faucet` per wallet per cooldown; by decision it stays enabled in the deferred mainnet deploy, since CFP has no redemption path. A win pays 1.98x (the house keeps 2% of the 1x profit); on a loss the wager stays in the contract: 4% goes to the house balance, a

fixed VRF fee to VRF compensation, and the remainder stays in the prize pool. `max_bet` derives from the token balance minus locked liability, fees and claimable credits.

3.2 Bet → VRF → payout, and the refund fence

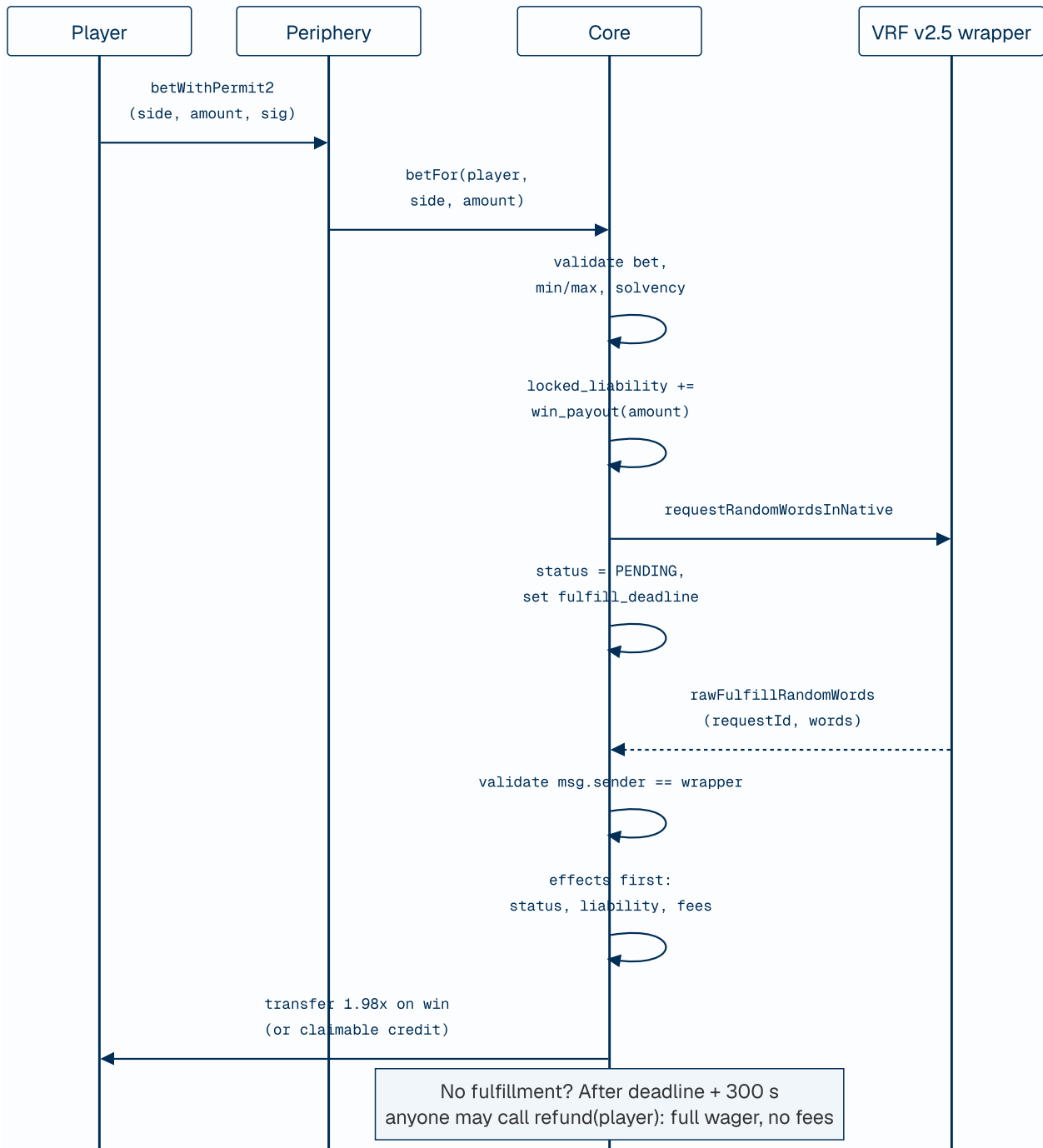


FIGURE 3 · BET → VRF → PAYOUT, AND THE REFUND FENCE

Resolution is CEI-ordered, a failing push transfer credits a `claim()`-able balance while the fulfillment still completes, and a callback for an unknown, expired or already-resolved bet is a no-op plus an event — a revert there would burn the Chainlink request and strand the bet. The refund path is permissionless and time-based, with a fixed, non-owner-tunable 300-second dead buffer so the fulfillment and refund windows never overlap.

3.3 Permit2 and ERC-1271

Inside Lemon, `CFP.approve(core) + core.bet` failed with `EXTERNAL_SERVICE_ERROR`, the same error Lemon returns for allowlist rejections, so CoinFlip took the Permit2 path through the periphery. BananitaSwap and Vaulty both run plain approve + call inside Lemon with the token allowlisted, so per-address allowlisting, not a smart-account limitation, is the operative variable. That account signs an **ERC-1271 permit of 162 bytes**, which the periphery's rigid `signature.len() != 65` check rejected with `InvalidSignatureLength` before Permit2 saw it. Dropping the shape check lets Permit2 dispatch ECDSA or `isValidSignature` itself, at the cost of a periphery redeploy to `0x5af619bf86de21f18f0fe3e671a2cb17f39bc931` (Sepolia, 2026-07-24, 18.9 KB).

3.4 Status

Public repository: `github.com/wakeuplabs/ArbitrumMiniApp-2-verifiable-randomness`, published 2026-08-13 as a reference implementation of the asynchronous-randomness pattern (Stylus contracts, Chainlink VRF v2.5, locked-liability accounting, permissionless refund); the `coinflip / cfp-*` crate names are kept as historical identifiers.

Arbitrum Sepolia — deployed and wired (mainnet-topology redeploy 2026-07-21; periphery redeployed 2026-07-24 for ERC-1271):

CONTRACT	ADDRESS	SIZE	ARBISCAN-VERIFIED
cfp-token	0x7e5638a1d198e9864e6e98eed1ecd1c1608bb980	15.0 KB / 1 fragment	yes
cfp-faucet	0xbc5f12dfa9d21cb99db11ef721c03c54e103220f	8.3 KB / 1 fragment	yes
coinflip core	0x55a52dc70e959fe85d862876ff624bb97dff96d3	23.7 KB / 1 fragment	yes
coinflip-periphery	0x5af619bf86de21f18f0fe3e671a2cb17f39bc931	18.9 KB / 1 fragment	—

Live QA covered every mandatory path with real transaction hashes: standalone approve+bet+VRF+payout, the Lemon betWithPermit2 path plus three Permit2 negatives, the AllowanceTransfer weekly-permit pair, views and access control, and a live refund. That refund needs a note on method: real VRF here resolves within ~30 s, so the callback was deliberately starved (`callback_gas_limit` lowered to 30,000, under the ~48k it needs) until the bet aged into the refund window and `refund(player)` returned exactly the 1 CFP wager — an *induced* timeout. **Arbitrum One is not deployed:** the core activates against Arbitrum One in cargo `stylus check` (one fragment), and the deployment itself was deferred by client decision. The ERC-1271 periphery redeploy used `--no-verify` (a native host build outside the reproducible Docker path), so the periphery now wired to the core is not Arbiscan-verified; the superseded periphery `0xb27fecfe906aca2b9a0219cec8b33e2aeb9c49ec` was.

4 Vault Aggregator (Vaulty)

4.1 Architecture

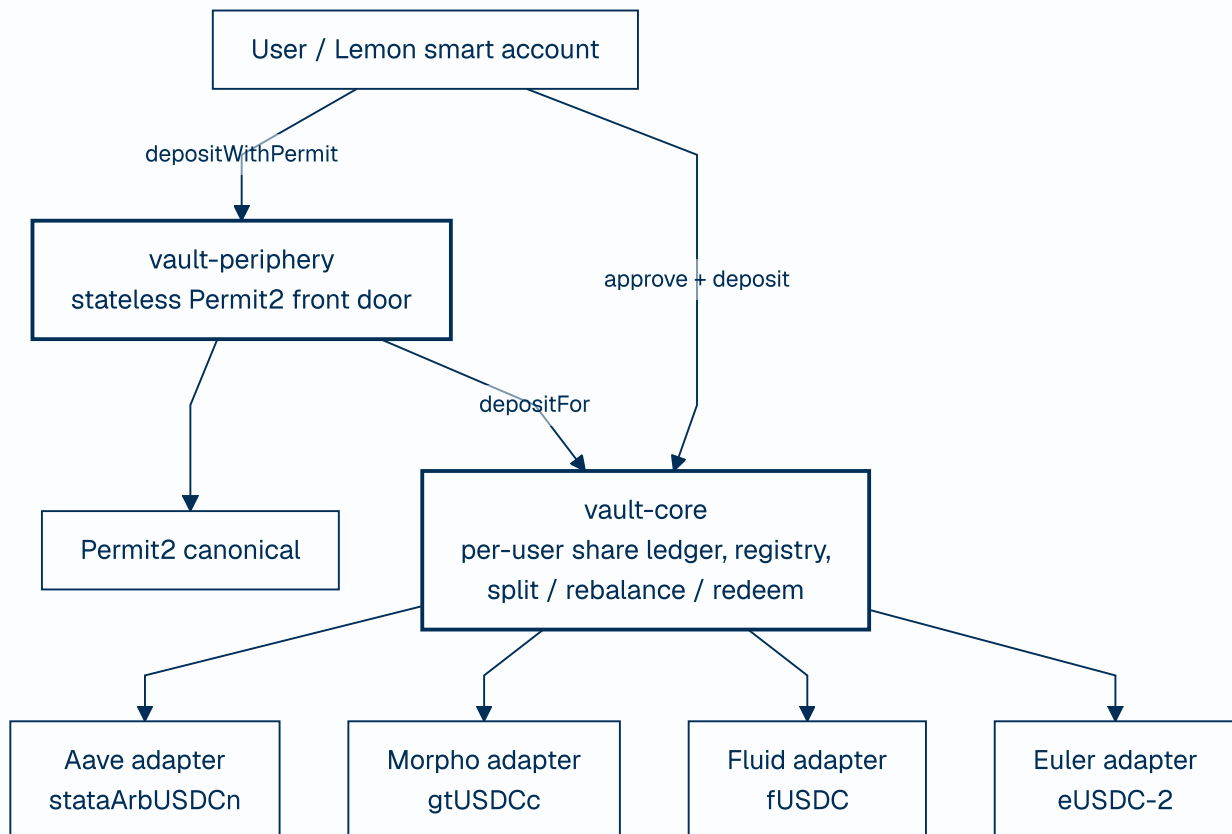


FIGURE 4 · ARCHITECTURE

vault-core is the system of record — per-user per-adapter share ledger, adapter registry, split/rebalance/redeem math — in a single `#[public] impl` block, since the SDK permits only one per contract type; vault-periphery is an optional stateless Permit2 front door; vault-adapter is one binary deployed once per protocol vault. The PRD's fourth protocol, Beefy, reverts on `asset()`, `convertToAssets()` and `maxRedeem()`, so it was swapped for Euler v2, collapsing the design to **one generic ERC-4626 adapter for all four**; Aave is reached through its official Stata wrapper, because the raw `aToken`'s `balanceOf` rebases upward with no transfer and breaks share accounting. Discovery also closed the PRD's central assumption: no deployed Stylus-native ERC-4626 router exists.

4.2 Deposit → yield → two-step withdraw

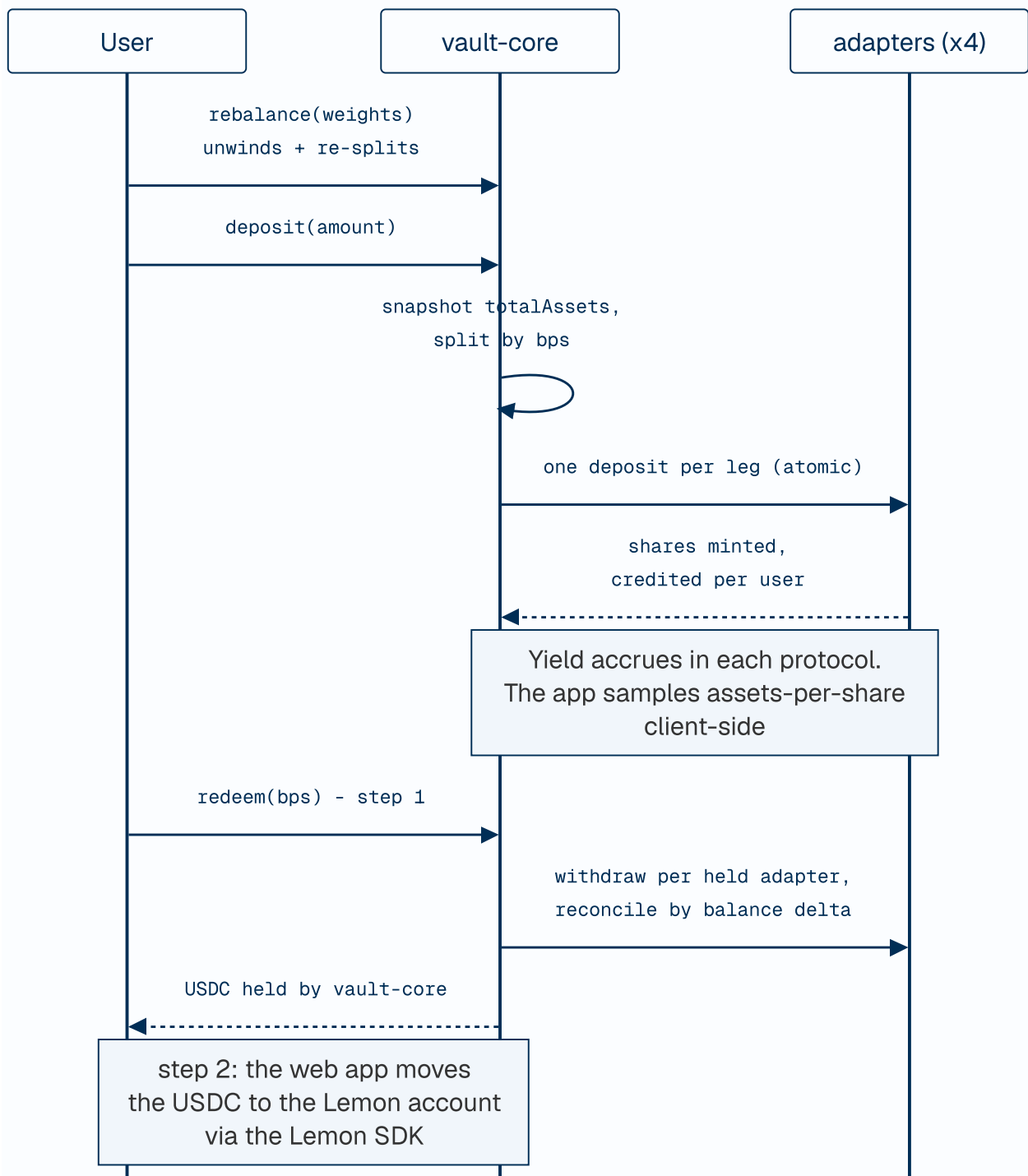


FIGURE 5 · DEPOSIT → YIELD → TWO-STEP WITHDRAW

Deposit is whole-transaction atomic across the caller's own stored weights, so a single failing leg reverts everything; rebalance is user-only and unwinds every adapter the caller holds shares in, so a disabled adapter can never strand them. Withdrawal is two steps by design — `redeem(bps)` returns USDC to vault-core, then the web app moves it to the Lemon account — over per-user, per-adapter share math with a virtual-offset floor-rounding scheme designed in from its first line.

4.3 Status: production on Arbitrum One

Public repository: `github.com/wakeuplabs/ArbitrumMiniApp-3-Vaulty`, published 2026-08-27 with the contracts, the web app and the full docs/ set this chapter draws on (runbook, size ledger, security review, known issues). Deployed 2026-08-10:

CONTRACT	ADDRESS
vault-core (owner = the deployer wallet)	0x8a1758d3dd3d1049c43bfb1d1fec11fd403d3553
vault-periphery	0x4e69892949f07623f3f59b24ebbb6e7ca2327bbc
Adapter — Morpho gtUSDCc	0x9aa8886c64d7b3799f676a41ad8bada77f128603
Adapter — Fluid fUSDC	0xe6f1d13787cb3cc8d0483fdd8a748f9d60ba5110
Adapter — Euler eUSDC-2	0x5ab92d390895ce6662568585413b42524eeabcfcd
Adapter — Aave stataArbUSDCn	0x156dc816990079917594cf1394389ed918e4ff3c

The mainnet smoke test ran a full real-USDC cycle the same day: approve → rebalance(40/30/20/10) → deposit(1 USDC), shares landing exactly 40/30/20/10 across the four protocols → rebalance(10/20/30/40) → redeem(10000 bps), at a net cost of **7 units** (7 millionths of a USDC) of ERC-4626 round-down with every `adapterTotalShares` back to 0. Earlier, before the production deploy, four interim adapter instances on Arbitrum One round-tripped **5/5 green** against the real protocols, losing another 7 units to round-down across those runs (14 in total), for a total interim-rig spend of **0.00069 ETH** (deploys, activations, round-trips and two out-of-gas failures with their recoveries). A disposable Arbitrum Sepolia rig runs the real core and adapters against four mock ERC-4626 vaults, and its own documentation is explicit that it proves the core's plumbing and **nothing** about the real protocols; QA there closed at 179/179 with a manual walkthrough including a real in-Lemon `depositWithPermit`. Arbiscan source verification is **pending**; the reason is in the mainnet-readiness chapter.

The mandated internal security checklist was cross-referenced against a **blind adversarial review** run with no access to it, the threat register or the known-issues file — a precaution earned by history, since a prior register had declared 22 threats closed while a code review then found a live exploitable defect in the exact invariant it claimed to protect. Its headline finding was HIGH and gating: `init(owner)` was front-runnable into permanent ownership, fixed with a real `#[constructor]`. Two known issues closed with measurement (0 dust across five cycles; exact two-user payout asserted on the rig) and one stands accepted: a throttled adapter reverts that user's whole redeem. A Lemon store submission kit is prepared; listing is outstanding, so there are **no production usage metrics**.

5 Stylus performance findings

5.1 The real mainnet gate: one fragment

The documented Stylus limit is 24 KB compressed. The operative limit for Arbitrum One is structurally different: **exactly one fragment**, because Arbitrum One's ArbOS version has no multi-fragment program support while Arbitrum Sepolia's does. Both projects therefore gate CI against the Arbitrum One endpoint; Vaulty additionally holds a self-imposed 22,528-byte (22 KB) threshold to keep ~1.5 KB of headroom under 24 KB against brotli non-determinism. CoinFlip's shipped core, at 23,761 bytes / 1 fragment, activates against Arbitrum One.

This was proven empirically. The same 2-fragment CoinFlip core binary:

- `cargo stylus check --contract coinflip --endpoint="https://arb1.arbitrum.io/rpc"`
→ **exit code 1**, error code 3: execution reverted, data: "0x"
- `cargo stylus check --contract coinflip --endpoint="https://sepolia-rollup.arbitrum.io/rpc"` → **exit code 0**

Sepolia tolerated the 2-fragment binary, which is why the existing CI gate had been silently passing.

5.2 Size levers, measured

CoinFlip's core started at **35,358 bytes / 2 fragments**. Each candidate was isolated on that baseline, applied, measured, and fully reverted before the next.

LEVER	MEASURED SAVING	IN THE SHIPPED CORE?
Move Permit2 intake to the periphery	1,711 bytes	yes
Demote three convenience view wrappers	151 bytes	yes
Collapse five admin setters into one router arm	225 bytes in isolation, ~450 once relocated to the periphery	yes
<code>wasm-opt -Oz</code> (Binaryen)	1,156 bytes / ~3.3%	no (declined for reproducibility)
Extract the whole VRF module into a third contract	2,700 bytes	no (kept as a documented fallback)

LEVER	MEASURED SAVING	IN THE SHIPPED CORE?
<code>-Cpanic=immediate-abort + -Zbuild-st</code> d	~9,900 bytes	yes, the decisive one

The arithmetic is the finding: the three structural moves plus `wasm-opt` left the core at **32,314 bytes / 2 fragments**, still rejected by Arbitrum One, and only eliminating the panic and `core::fmt` machinery closed the gate. The shipped core is that path without `wasm-opt` and without the VRF split: **23,761 bytes / 1 fragment**. The same spike put the per-contract `stylus-sdk` baseline at **~12-15 KB** — a near-minimal contract weighs 14.7 KB — so splitting a contract never divides its cost. Byte-identical source also varies by **~100-150 bytes** across builds from `broccoli` non-determinism, which is why a measured 53-byte fit for a `Permit2` intake inside `vault-core` was rejected and `vault-periphery` exists as its own contract.

Shipped sizes, all single-fragment: `coinflip` core 23.7 KB, `periphery` 18.9 KB, `cfp-token` 15.0 KB, `cfp-faucet` 8.3 KB, `vault-core` 20,351 B, `vault-adapter` 12,550 B (as checked on the source tree; the interim deployed instances measured 12,577-12,578 B), `vault-periphery` 10,266 B.

5.3 Activation cost

Activation is charged once per codehash: three adapter instances sharing one codehash paid a single fee of **0.000101 ETH**, the later deploys logging `wasm already activated!`. One instructive accident on the interim rig: the Aave adapter paid a second activation because its binary came out 12,578 bytes against 12,577 for an identical source tree, and one byte changes the codehash and therefore the `ArbWasm` cache key. `CoinFlip`'s activation check against Arbitrum One quoted **0.000116 ETH** for a 22.2 KB spike build of the core (one that carried a manual `wasm-opt` pass the production pipeline does not run).

5.4 Gas per operation

OPERATION	NETWORK	GASUSED
<code>CoinFlip bet</code> (standalone, triggers the VRF request)	Arbitrum Sepolia	431,573
<code>CoinFlip betWithPermit2</code> (Lemon path)	Arbitrum Sepolia	453,841
<code>CoinFlip core deploy + activation</code> (23.7 KB monolith)	Arbitrum Sepolia	10,596,141
<code>Vaulty deposit</code>	Arbitrum One	1,604,156
<code>Vaulty rebalance</code> (full unwind + re-split)	Arbitrum One	2,607,088
<code>Vaulty redeem</code>	Arbitrum One	1,741,442

Full per-transaction receipts (62 rows, read from chain) are kept in the companion gas-measurements file. `Permit2` intake costs about 22k gas over the classic approve path, so the signature-per-bet UX is close to free; `Vaulty`'s writes are expensive in absolute terms because

each fans out to four protocols atomically. The Stylus VRF callback itself costs **≈48k gas**, from a `gas_left` snapshot inside a real fulfillment showing 452,387 remaining of a provisional 500,000 limit.

5.5 `eth_estimateGas` under-reports Stylus→EVM calls

Two live Arbitrum One round-trips on the interim adapter rig failed on their first pass with **out-of-gas**: each was sent with an estimator-derived limit and mined at `status=0` with `gasUsed == gasLimit`, while the identical call replayed through `eth_call` returned successfully, ruling out a logic revert. Retried with a generous explicit limit, each succeeded, using less gas than the limit that had run out.

VAULT	ETH_ESTIMATEGAS	FAILED TX: GASUSED = GASLIMIT	SUCCESSFUL RETRY: GASUSED
Euler	217,692	212,473	198,382
Morpho	602,149	596,113	578,240

The receipts do not explain the gap between what the failed attempt burned and what the successful one needed; what they establish is that a limit at or near the estimator's number is not safe for a Stylus call that fans out to EVM protocols, and that headroom fixes it. The test harness pinned 2,000,000 on every mutating call; the production frontend and scripts pin **6,000,000** after a 2,000,000 limit OOG'd the first real mainnet rebalance. Unused gas refunds on Arbitrum, so the headroom is free — and the runbook's own warning generalizes it: a wagmi/viem `writeContract` on automatic estimation produces exactly this failure mode, and **it presents as a contract bug**.

5.6 Randomness funding: native ETH over LINK

The choice of native ETH over LINK for Chainlink VRF is backed by arithmetic: LINK's lower premium (50% vs 60%) is offset by +22,000 gas of coordinator overhead, so below a `callbackGasLimit` of roughly 212k native wins by 3-5%, and the measured ~48k callback sits firmly in that zone.

6 Integration patterns

6.1 Chainlink VRF v2.5 direct funding from Stylus

What we did. Called the VRF v2.5 wrapper via `sol_interface!` and exported a hand-written `rawFulfillRandomWords(uint256,uint256[])` callback with a hand-written sender check, since Stylus has no `VRFConsumerBaseV2Plus` to inherit.

What bit us. In direct-funding mode the **wrapper** performs the callback; the coordinator never calls the consumer. Validating `msg.sender == coordinator` made every fulfillment revert with `WrapperFulfillmentFailed` while the coordinator reported `success = true` — a signal that reads as "Chainlink never fulfills" when it was fulfilling within ~30 s.

What to do. Validate against the wrapper, mirroring `VRFV2PlusWrapperConsumerBase.onlyVRFWrapper`, and encode `extraArgs` exactly as `bytes4(keccak256("VRF ExtraArgsV1")) ++ abi.encode(nativePayment=true)`.

6.2 `sol_interface!` computes the wrong selector for struct-shaped calldata

What we did. Bound external protocol interfaces (Permit2, Aave, Morpho) from Stylus.

What bit us. `sol_interface!` computes the wrong function selector for calldata parameters shaped as structs — `build_selector` uses the literal type name where the canonical form is a flattened tuple, proven byte-for-byte against `cast sig` on CoinFlip's Permit2 intake. Aave, Morpho and Permit2 all expose struct-parameter entrypoints, so the defect silently miscalls them.

What to do. Use `alloy_sol_types::sol!` bare function signatures with manual dispatch for any such binding, and cross-check every selector against `cast sig` before freezing the adapter — ERC-4626 `deposit 0x6e553f65` and `redeem 0xba087652` (what the shipped adapters call, Aave included via its Stata wrapper), plus Aave Pool `supply 0x617ba037` and `withdraw 0x69328dec`, probed during discovery before the Stata decision.

6.3 TestVM's single return-data buffer

What we did. Used `stylus-test`'s TestVM for the bulk of unit testing on both projects.

What bit us. `stylus-test` 0.10.8 keeps **one** return-data buffer, set at *mock-registration* time and shared by every call in the frame, so inside a single top-level call every mocked external call

decodes the same bytes. In `vault-core` this collapsed the two `balanceOf(core)` reads inside `redeem` and made a whole branch unreachable: forcing `maxWithdraw() == 0` forces `totalAssets() == 0` in the same call, flooring the owed amount to zero, so the full-throttle case and the real bug hiding in it were invisible by construction.

What to do. Chain external calls as separate statements invoking private methods, with fresh mocks between them. Accept that some assertions are unreachable under `TestVM` and move those to a live testnet rig.

6.4 Permit2 from Stylus, and ERC-1271 signatures

What we did. Both projects pull tokens through canonical `Permit2` from a stateless periphery, with the destination set to the core.

What bit us. A rigid 65-byte length check rejected Lemon's 162-byte ERC-1271 signature before `Permit2` could evaluate it. An earlier investigation had also blamed a Stylus bytes parameter combined with a nested call for empty reverts; a ten-variant probe matrix across six deploys failed to reproduce that on `stylus-sdk = 0.10.7`, and the real cause was a one-hex-digit typo in a hardcoded address.

What to do. Let `Permit2` validate signatures itself; it dispatches ECDSA or ERC-1271 correctly. When a Stylus call reverts with empty data, suspect a wrong address or selector before suspecting the SDK.

6.5 Lemon `callSmartContract` and native `Permit2`

What we did. Ran both mini-apps inside Lemon's mobile `WebView`, where `callSmartContract` has **Permit2 built in**: signed permits replace separate approve transactions.

What bit us. A mid-program conclusion held that Lemon could not do `Permit2` signature substitution, and `vault-periphery` was deleted on that basis. The failure was in a hand-rolled server-side scheme. `Permit2` inside Lemon works, natively signing a real `SignatureTransfer EIP-712` struct client-side via `permits[]`, so the periphery was restored.

What to do. Treat "the platform cannot do X" as a hypothesis with an expiry date when the evidence is one failed implementation. Keep the plain `approve + call` path working regardless: standalone browsers need it, and inside Lemon it works once the token is allowlisted (Vaulty and BananitaSwap ship it that way).

7 Mainnet-readiness

7.1 The pipeline that worked

`cargo stylus check --endpoint="https://arb1.arbitrum.io/rpc"` runs as a CI gate from the first crate, with a per-task STOP rule: measure after every task touching core source, halt above one fragment (Vaulty also halts above its 22,528-byte working threshold), escalate for an explicit decision. Deploys go through the reproducible Docker path followed by `cargo stylus verify`, shipping the same bytecode to every network with no compile-time features. Mainnet sits behind a written pre-flight list: tag the source, `cargo test --workspace green`, re-run every size gate against Arbitrum One, security checklist sign-off, re-probe the four protocol vaults read-only, confirm the pinned nightly and Docker are intact.

7.2 The pipeline's open problem: verification of `-Zbuild-std` builds

`cargo stylus verify`'s reproducible Docker build does not reproduce a `-Zbuild-std` artifact with the stock image. Since Arbiscan source verification is a deliverable and the nightly recipe is the only lever that fits each core under the mainnet gate, the two requirements are in direct tension, and it binds both projects: CoinFlip's core and every Vaulty binary are nightly / `-Zbuild-std` builds. The workaround — a custom published build image pinned by digest — works and is how CoinFlip's core was verified on Sepolia, but it asks every third party wanting to reproduce the bytecode to trust and pull a non-official image. `wasm-opt` was **declined** in Vaulty for the same reason, despite being a free ~3.3%: it adds a transform a third party must replicate exactly.

7.3 Fork testing: the state of the tooling

Both candidates were tested to exhaustion against one literal bar — a Stylus contract making a real call against forked Aave state:

- **nitro-testnode: no fork mechanism exists.** Confirmed by enumerating the full flag surface of the 825-line `test-node.bash` on the release branch; every flag configures a from-genesis devnode, and the most-cited third-party fork is a stale mirror with a byte-identical README.
- **TestVMBuilder:::rpc_url(): partial, and the partial does not help.** `stylus-test` 0.10.8's source forks only the storage of the TestVM's own `contract_address`; external calls always route through the local mock registry regardless of `rpc_url`, proven by a static call to Aave Pool from a different address returning an empty, unmocked success. An adapter's whole job is calling out to the four protocols, exactly the shape TestVM never forks.

The substitute that shipped: small real-USDC transactions against the real protocols on Arbitrum One, adopted as the frozen replacement for fork tests. It works — 5/5 round-trips green, real protocol quirks surfaced — but it costs real funds and cannot be run by a contributor without a funded mainnet wallet.

7.4 Immutable owner, by design

`vault-core` has **no `transferOwnership` on its ABI**: the constructor's `owner` runs once at deploy and owns `addAdapter/setEnabled` forever, so redeploy is the ownership-change path. It replaced an earlier `init(owner)` the blind review found front-runnable: in Stylus, a post-deploy `init` that sets ownership is a claimable entrypoint.

7.5 Recommendations to the Stylus / Arbitrum team

1. **Bring multi-fragment support to Arbitrum One**, or make `cargo stylus check` warn when the target endpoint's ArbOS would reject the fragment count just measured.
2. **Fix `eth_estimateGas` for Stylus→EVM calls** — a limit taken from the estimator running out of gas on a call that later succeeds using less than that limit is a correctness bug in the estimator.
3. **Make `cargo stylus verify reproduce -Zbuild-std` artifacts**, or drop `lazy_static/std` from `stylus-core` so a plain stable `no_std` build reaches the same size.
4. **Document callback-selector expectations for oracle integrations** — the plumbing-vs-hook distinction is free in Solidity via inheritance and invisible in Stylus.
5. **Provide a fork-testing story**, via an import-state mode for `nitro-testnode` or by extending `TestVM`'s RPC forking to external calls.

One smaller tooling bug hit live: `cargo stylus verify --deployment-tx` is broken in multi-crate workspaces, because `verify`'s constructor reflection runs `cargo run` without `--package`.